LAPORAN PRAKTIKUM UJIAN AKHIR SEMESTER

KOMPUTASI STATISTIKA II

KELAS B

Yogyakarta, 30 Juni 2024

Anggota kelompok	: Dwi Cahyawati	(22/498356/PA/21506)
	Rhizka Febrianty Aisyah Dewi	(22/503643/PA/21652)
	Marta Afifah	(22/504888/PA/21725)
Dosen Pengampu	: Prof. Dr.rer.nat. Dedi Rosadi, S.Si., M.Sc.	
Asisten Praktikum	: Ainur Rokhimah (20596)	
	Raditya Arviandana (20632)	

DEPARTEMEN MATEMATIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS GADJAH MADA

2024

BAB I
PERMASALAHAN

Sebagai negara yang terletak di sekitar Ring Of Fire, tak jarang negara kita, Indonesia, mengalami bencana gempa bumi baik akibat dari pergerakan lempeng tektonik maupun aktivitas gunung berapi. Dengan tingginya angka bencana ini, tentunya tidak sedikit kerugian yang didapat. Guna mengantisipasi terjadinya bencana gempa bumi kedepannya, sebuah perusahaan properti ingin mengetahui pengaruh dari spesifikasi bangunan terhadap kerusakan yang diterima oleh bangunan tersebut. Maka dari itu, dengan dataset yang diberikan lakukanlah analisis untuk memprediksi dampak yang diterima oleh suatu bangunan berdasarkan spesifikasinya

BAB II
PEMBAHASAN

2.1 Penjelasan Dataset

Dataset yang digunakan terdiri dari 24 fitur dan 310009 data. Berikut adalah penjelasan dari tiap fitur:

No	Nama Fitur	Penjelasan
1.	floors_before_eq (total)	jumlah total lantai pada bangunan sebelum gempa terjadi
2.	old_building	usia bangunan (tahun)
3.	plinth_area (ft^2)	luas bangunan (ft^2)
4.	height_before_eq (ft)	tinggi bangunan sebelum terjadi gempa (kaki)
5.	land_surface_condition	kondisi permukaan tanah disaat bangunan akan dibangun
6.	type_of_foundation	jenis fondasi yang dipakai untuk bangunan tsb
7.	type_of_roof	jenis atap yang dipakai untuk bangunan tsb
8.	type_of_ground_floor	jenis lantai yang dipakai untuk ground-floor
9.	type_of_other_floor	jenis lantai yang dipakai untuk selain ground-floor
10.	position	posisi bangunan tersebut apakah bersisian secara langsung dengan bangunan lain
11.	building_plan_configuration	konfigurasi bangunan yang berkaitan dengan bentuk, ukuran, dan penempatan struktur utama bangunan
12.	technical_solution_proposed	solusi yang ditawarkan untuk bangunan yang terdampak gempa
13.	legal_ownership_status	status kepemilikan bangunan
14.	has_secondary_use	keterangan apakah bangunan memiliki kegunaan sekunder
15.	type_of_reinforcement_concrete	tipe beton bertulang/reinforcement concrete (0: No reinforcement concrete, 1: Has non-engineered reinforcement concrete, 2: Has engineered reinforcement concrete, 3: Has both)
16.	residential_type	tipe penggunaaan sebagai hunian
17.	no_family_residing	jumlah keluarga yang tinggal dalam bangunan tersebut
18.	public_place_type	tipe penggunaan sebagai tempat umum
19.	industrial_use_type	tipe industri
20.	govermental_use_type	tipe penggunaan sebagai bangunan pemerintahan

21.	flexible_superstructure	keterangan penggunaan superstructure yang fleksibel
22.	wall_binding	material yang digunakan sebagai perekat bahan pembentuk dinding (0: Unknown/not stated, 1: Clay, 2: Mortar+Cement, 3: Mortar + Cement, Clay, 5: Mud + Mortar, Clay, 7: Mud + Mortar, Clay, Cement + Mortar)
23.	wall_material	material dasar sebagai pembangun dinding (0: Unknown/not stated, 1: Red Bricks, 2: Stone Bricks, 3: Red Bricks, Stone Bricks)
24.	damage_grade (variabel target)	tingkat kerusakan yang disebabkan oleh gempa (1-5)

2.2 Analisis

2.2.1 Import Dataset

Langkah awal yang perlu dilakukan adalah pengimporan dataset yang akan digunakan dalam analisis. Dalam kasus ini terdapat 3 dataset yang digunakan dalam format CSV, yaitu sebagai berikut :

- Data fitur train

```
features = pd.read_csv("DAta/Dataset
Responsi/train_features.csv")
features
```

	ID	floors_before_eq (total)	old_building	plinth_area (ft^2)	height_before_eq (ft)	land_surface_condition	type_of_foundation	type_of_roof	type_of_ground_floor	type_of_other_floor	...	has_secondary_use	type_of_reinforcement_concret
0	5054	Floor 3	26.0	300 ft^2	16.0	Flat	Clay mortar-Stone/Brick	Wood Light Roof or Bamboo Heavy Roof	Clay	Timber/Bamboo-Mud	...	0.0	0
1	166563	There is 2 Floor/Story	16.0	263 ft^2	18.0	Moderate slope	Mud mortar-Stone/Brick	Bamboo/Timber-Light Roof	mud	Timber/Bamboo-Mud	...	0.0	0
2	25330	Floor two	5.0	NaN	10.0	Flat	Clay Sand Mixed mortar-Stone/Brick	NaN	mud	Timber-Planck	...	0.0	0
3	60282	NaN	27.0	NaN	NaN	Flat	Clay Sand Mixed mortar-Stone/Brick	NaN	NaN	Timber/Bamboo-Mud	...	1.0	0
4	194414	1	6.0	380 ft^2	6.0	Moderate slope	Clay mortar-Stone/Brick	Bamboo or Timber Light roof	Clay	Not applicable	...	0.0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	
310004	253891	just 2 floor	6.0	192 ft^2	14.0	Flat	Clay mortar-Stone/Brick	Bamboo/Timber-Heavy Roof	Clay	rcc/rb/rbc	...	1.0	0

- Data label train

```
label = pd.read_csv("DAta/Dataset Responsi/train_label.csv")
label
```

	ID	damage_grade
0	5054	5
1	166563	5
2	25330	2
3	60282	5
4	194414	4
...	...	...
310004	253891	4
310005	4279	5
310006	352159	5
310007	257848	5
310008	212492	5

310009 rows × 2 columns

- Data fitur test

```
test = pd.read_csv("DAta/Dataset Responsi/test_features.csv")
test
```

	ID	floors_before_eq (total)	old_building	plinth_area (ft^2)	height_before_eq (ft)	land_surface_condition	type_of_foundation	type_of_roof	type_of_ground_floor	type_of_other_floor	...	has_secondary_use	type_of_reinforcement_concrete
0	71840	floor third	20.0	374 ft^2	21.0	Flat	Clay mortar-Stone/Brick	Wood Light Roof or Bamboo Light Roof	Clay	Timber/Bamboo-Mud	...	0.0	0.0
1	85155	Floor two	26.0	364 ft^2	16.0	Flat	Clay mortar-Stone/Brick	Wood Light Roof or Bamboo Light Roof	Clay	Timber/Bamboo-Mud	...	0.0	0.0
2	287066	NaN	6.0	NaN	NaN	NaN	Clay mortar-Stone/Brick	NaN	NaN	NaN	...	1.0	0.0
3	124951	There is 2 Floor/Story	25.0	345 ft^2	12.0	Moderate slope	Clay mortar-Stone/Brick	Wood Light Roof or Bamboo Light Roof	Clay	Wood-Plank	...	0.0	0.0
4	73581	Three	25.0	324 ft^2	14.0	Moderate slope	Clay mortar-Stone/Brick	Wood Light Roof or Bamboo Heavy Roof	Mud	Lumber-plank	...	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
77498	238185	Floor two	17.0	NaN	15.0	Moderate slope	Clay mortar-Stone/Brick	NaN	Clay	Timber/Bamboo-Mud	...	0.0	0.0

Selanjutkan dilakukan penggabungan data fitur train dan label train.

```
train = pd.merge(features, label, on='ID')
train

train.describe()

train.info()
```

Didapatkan data train dengan deskripsi dan tipe data sebagai berikut :

	ID	old_building	height_before_eq (ft)	has_secondary_use	type_of_reinforcement_concrete	no_family_residing	wall_binding	wall_material	damage_grade
count	310009.000000	307150.000000	282223.000000	308620.000000	295248.000000	279243.000000	309981.000000	307736.000000	310009.000000
mean	193624.837266	24.495667	16.043218	0.127830	0.073047	1.087483	4.231379	1.729700	3.615056
std	111884.480023	66.207343	5.520064	0.333901	0.320298	0.349594	1.701756	0.656351	1.350309
min	0.000000	0.000000	6.000000	0.000000	0.000000	1.000000	0.000000	0.000000	1.000000
25%	96722.000000	8.000000	12.000000	0.000000	0.000000	1.000000	5.000000	2.000000	3.000000
50%	193567.000000	16.000000	16.000000	0.000000	0.000000	1.000000	5.000000	2.000000	4.000000
75%	290529.000000	27.000000	18.000000	0.000000	0.000000	1.000000	5.000000	2.000000	5.000000
max	387511.000000	999.000000	99.000000	1.000000	3.000000	11.000000	7.000000	3.000000	5.000000

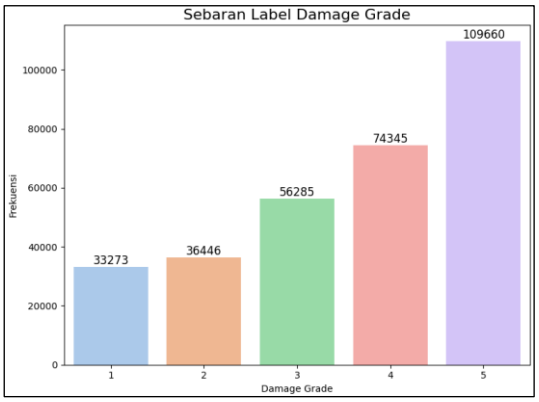
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 310009 entries, 0 to 310008
Data columns (total 25 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   ID                              310009 non-null  int64
 1   floors_before_eq (total)        282223 non-null  object
 2   old_building                    307150 non-null  float64
 3   plinth_area (ft^2)              240977 non-null  object
 4   height_before_eq (ft)           282223 non-null  float64
 5   land_surface_condition          292578 non-null  object
 6   type_of_foundation              307150 non-null  object
 7   type_of_roof                    240977 non-null  object
 8   type_of_ground_floor            282223 non-null  object
 9   type_of_other_floor             292578 non-null  object
 10  position                        289075 non-null  object
 11  building_plan_configuration     292578 non-null  object
 12  technical_solution_proposed     37406 non-null   object
 13  legal_ownership_status          309806 non-null  object
 14  has_secondary_use               308620 non-null  float64
 15  type_of_reinforcement_concrete  295248 non-null  float64
 16  residential_type                299882 non-null  object
 17  no_family_residing              279243 non-null  float64
 18  public_place_type               310009 non-null  object
 19  industrial_use_type             309909 non-null  object
 20  govermental_use_type            304469 non-null  object
 21  flexible_superstructure         309981 non-null  object
 22  wall_binding                    309981 non-null  float64
 23  wall_material                   307736 non-null  float64
 24  damage_grade                    310009 non-null  int64
dtypes: float64(7), int64(2), object(16)
memory usage: 59.1+ MB
```

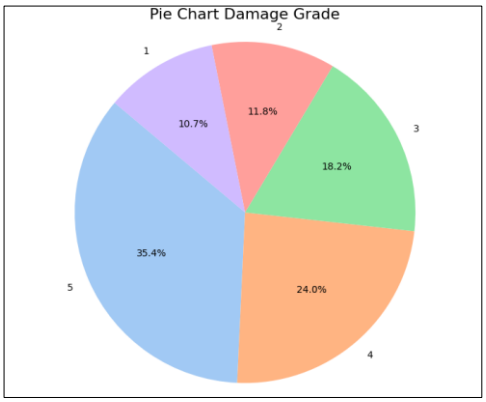
Data train memiliki sebanyak 24 variabel dengan 23 variabel diantaranya adalah variabel fitur dan variabel '*damage_grade*' sebagai variabel respon. Variabel '*damage_grade*' menunjukkan tingkat kerusakan bangunan akibat gempa dengan nilai 1 s.d 5. Variabel ini berfungsi sebagai target dalam analisis klasifikasi untuk memprediksi tingkat kerusakan berdasarkan fitur-fitur yang tersedia.

2.2.2 *Exploratory Data Analysis* (EDA)

Tahap EDA dilakukan untuk memperoleh informasi yang tersimpan dalam data. Berikut merupakan EDA yang dilakukan pada dataset yang diberikan.

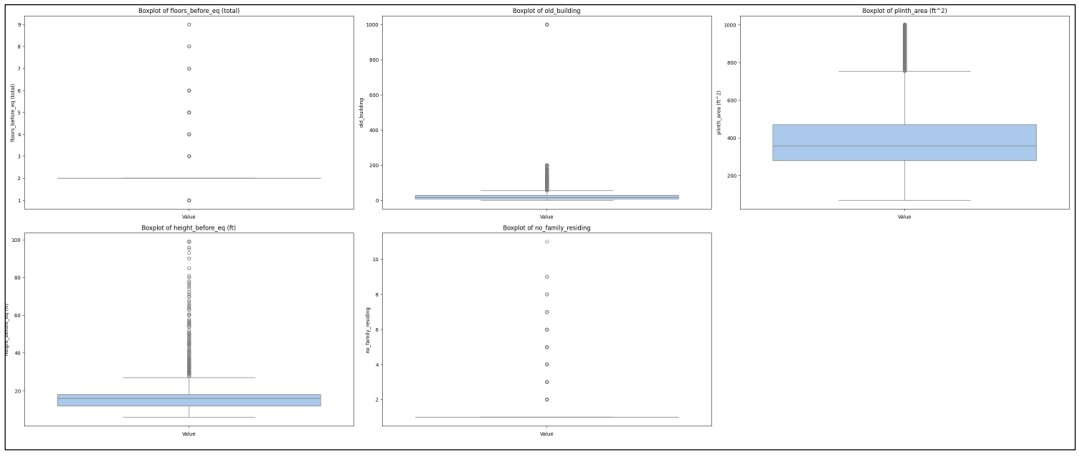
Barchart dan Pie Chart Label Damage Grade

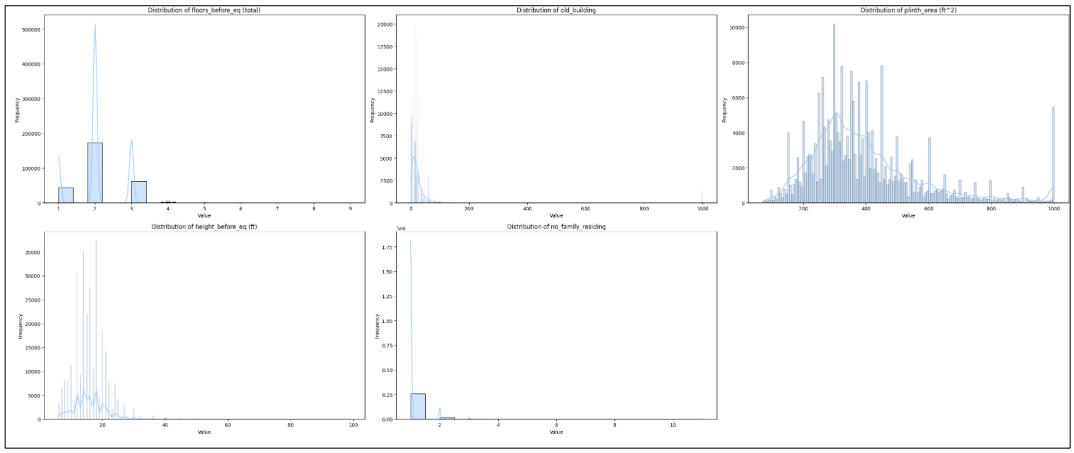




Berdasarkan grafik batang dan diagram lingkaran, terlihat bahwa tingkat kerusakan parah mendominasi. Kerusakan tingkat 5, yang paling berat, mencakup 35,4% dari total kasus dengan 109.660 insiden, diikuti oleh tingkat 4 dengan 24% atau 74.345 kasus. Kerusakan tingkat menengah (tingkat 3) mewakili 18,2% dengan 56.205 kasus. Sementara itu, kerusakan ringan (tingkat 1 dan 2) hanya mencakup sebagian kecil, masing-masing 11,8% (33.773 kasus) dan 10,7% (36.446 kasus). Secara keseluruhan, lebih dari 59% struktur mengalami kerusakan berat (tingkat 4-5), sementara kerusakan ringan hingga sedang (tingkat 1-3) mencakup sekitar 41% kasus. Distribusi ini mengindikasikan bahwa gempa tersebut menyebabkan dampak yang sangat serius, dengan mayoritas bangunan mengalami kerusakan parah.

Boxplot dan Histogram Variabel Numerik Data Train





- ’*Floors_before_eq*’ (Jumlah lantai bangunan sebelum gempa): Mayoritas bangunan memiliki 2-3 lantai, dengan puncak tertinggi pada 2 lantai. Bangunan 1 dan 4 lantai juga cukup umum. Boxplot menunjukkan adanya outlier hingga 9 lantai, namun histogram mengonfirmasi bahwa bangunan di atas 4 lantai sangat jarang.
- ’*Old_building*’ (Usia bangunan): Distribusi usia bangunan sangat miring ke kanan. Sebagian besar bangunan berusia relatif muda, namun boxplot menunjukkan adanya outlier yang mengindikasikan beberapa bangunan sangat tua. Histogram memperjelas bahwa mayoritas bangunan berusia di bawah 100 tahun, dengan penurunan tajam untuk usia yang lebih tua.
- ’*Plinth_area*’ (Luas alas bangunan): Distribusi luas alas bangunan cenderung normal namun sedikit miring ke kanan. Boxplot dan histogram sama-sama menunjukkan variasi yang cukup lebar, dengan mayoritas bangunan memiliki luas antara 200-600 m². Terdapat outlier yang mencolok, mengindikasikan beberapa bangunan dengan luas yang sangat besar.
- ’*Height_before_eq*’ (Tinggi bangunan sebelum gempa): Selaras dengan jumlah lantai, mayoritas bangunan memiliki tinggi yang relatif seragam antara 10-30 kaki. Boxplot menampilkan outlier yang signifikan, sementara histogram memperlihatkan penurunan tajam frekuensi untuk bangunan yang lebih tinggi.
- *’No_family_residing*’ (Jumlah keluarga yang tinggal): Boxplot dan histogram konsisten menunjukkan bahwa hampir semua bangunan dihuni oleh 1 keluarga. Boxplot memperlihatkan outlier, yang dikonfirmasi oleh histogram bahwa ada sedikit sekali bangunan yang dihuni oleh 2 atau lebih keluarga.

2.2.3 Preprocessing

2.2.3.1 Inkonsistensi Data

Mengubah semua kata dalam data menjadi huruf kecil (*lowecase*)

Penanganan pada tahap ini diterapkan pada dataset train dan dataset test.

```
# Mengubah setiap sel menjadi lowercase
train = train.applymap(lambda x: x.lower() if isinstance(x,
str) else x)
test = test.applymap(lambda x: x.lower() if isinstance(x, str)
else x)
```

Variabel '*floor_before_eq*'

Diketahui bahwa variabel '*floor_before_eq*' pada dataset train dan test memiliki ketidakkonsistenan dalam representasi data. Beberapa entri menggunakan angka, sementara yang lain menggunakan kata-kata atau frasa untuk menggambarkan jumlah lantai.

```
display(train['floors_before_eq (total)'].unique(),
test['floors_before_eq (total)'].unique())
```

```
array(['floor 3', 'there is 2 floor/story', 'floor two', nan, '1',
       'has 1 floor', ' has 3 floor', 'one', 'three', 'floor 1',
       ' just 2 floor', 'two', 'five floor', '2 floor', '2',
       'floor second', '3', 'two floor', 'floor 1st', 'two story',
       'three story', 'three floor', 'floor 2', 'floor-three',
       'just 1 floor', 'floor one', '5', 'one story', '1st story',
       'four story', 'four floor', 'floor third', 'floor four', '4',
       ' has four fl', 'fl four', 'floor 4', 'four', ' has five fl',
       'fifth', 'floor fifth', 'fl five', '6', '7', '8', '9',
       'has five fl'], dtype=object)
array(['floor third', 'floor two', nan, 'there is 2 floor/story', 'three',
       '2 floor', 'has 1 floor', '3', 'two floor', 'floor 1st',
       'floor-three', 'two story', 'three story', 'two', 'floor 2',
       'one story', '2', 'three floor', 'floor one', ' just 2 floor',
       'fl four', '1st story', ' has 3 floor', 'floor second', '4',
       'just 1 floor', 'one', 'floor 3', '1', 'floor 1', 'floor fifth',
       'fifth', 'floor 4', 'fl five', 'floor four', 'four floor', '6',
       'five floor', '5', 'four story', 'four', ' has four fl', '8',
       ' has five fl', '7', '9'], dtype=object)
```

Sehingga dilakukan penanganan dengan menggunakan kamus (*dict_floors_before_eq*) yang memetakan berbagai representasi ke nilai numerik yang sesuai. Proses penanganan ini diterapkan pada kedua dataset menggunakan fungsi *apply* dan *lambda*, mengkonversi setiap entri ke format numerik yang seragam.

```
dict_floors_before_eq = {
    'floor 3': 3,
    'there is 2 floor/story': 2,
    'floor two': 2,
    '1': 1,
    'has 1 floor': 1,
    ' has 3 floor': 3,
    'one': 1,
    'three': 3,
    'floor 1': 1,
    ' just 2 floor': 2,
    'two': 2,
    'five floor': 5,
    '2 floor': 2,
    '2': 2,
    'floor second': 2,
    '3': 3,
    'two floor': 2,
    'floor 1st': 1,
    'two story': 2,
    'three story': 3,
    'three floor': 3,
```

```
    'floor 2': 2,
    'floor-three': 3,
    'just 1 floor': 1,
    'floor one': 1,
    '5': 5,
    'one story': 1,
    '1st story': 1,
    'four story': 4,
    'four floor': 4,
    'floor third': 3,
    'floor four': 4,
    '4': 4,
    ' has four fl': 4,
    'fl four': 4,
    'floor 4': 4,
    'four': 4,
    ' has five fl': 5,
    'fifth': 5,
    'floor fifth': 5,
    'fl five': 5,
    '6': 6,
    '7': 7,
    '8': 8,
    '9': 9,
    'has five fl': 5
}

train['floors_before_eq (total)'] = train['floors_before_eq
(total)'].apply(lambda x: dict_floors_before_eq.get(x, x))
test['floors_before_eq (total)'] = test['floors_before_eq
(total)'].apply(lambda x: dict_floors_before_eq.get(x, x))
display(train['floors_before_eq (total)'].value_counts(),
test['floors_before_eq (total)'].value_counts())
```

Sehingga dihasilkan output sebagai berikut:

```
floors_before_eq (total)
2.0    172850
3.0     61941
1.0     44470
4.0      2229
5.0       569
6.0       121
7.0        32
8.0         7
9.0         4
Name: count, dtype: int64
floors_before_eq (total)
2.0    43415
3.0    15608
1.0    10921
4.0      610
5.0      151
6.0       33
7.0        4
9.0        2
8.0        1
Name: count, dtype: int64
```

Variabel '*plinth_are(ft^2)*'

Variabel 'plinth_area (ft^2)' pada dataset train dan test memiliki ketidakkonsistenan dalam representasi data.

```
display(train['plinth_area (ft^2)'].unique(), test['plinth_area
(ft^2)'].unique())
```

```
array(['300 ft^2', '263 ft^2', nan, '380 ft^2', '155 ft^2', '312 ft^2',
       '460 ft^2', '250 ft^2', '350 ft^2', '400 ft^2', '875 ft^2',
       '383 ft^2', '453 ft^2', '431 ft^2', '162 ft^2', '540 ft^2',
       '750 ft^2', '264 ft^2', '292 ft^2', '240 ft^2', '385 ft^2',
       '384 ft^2', '275 ft^2', '405 ft^2', '370 ft^2', '203 ft^2',
       '306 ft^2', '320 ft^2', '650 ft^2', '360 ft^2', '700 ft^2',
       '342 ft^2', '450 ft^2', '200 ft^2', '500 ft^2',
       'more than 1000 ft^2', '420 ft^2', '390 ft^2', '458 ft^2',
       '812 ft^2', '481 ft^2', '924 ft^2', '900 ft^2', '252 ft^2',
       '445 ft^2', '666 ft^2', '520 ft^2', '511 ft^2', '351 ft^2',
       '626 ft^2', '260 ft^2', '368 ft^2', '268 ft^2', '295 ft^2',
       '99 ft^2', '186 ft^2', '600 ft^2', '480 ft^2', '336 ft^2',
       '438 ft^2', '496 ft^2', '324 ft^2', '225 ft^2', '190 ft^2',
       '428 ft^2', '565 ft^2', '231 ft^2', '546 ft^2', '142 ft^2',
       '265 ft^2', '412 ft^2', '120 ft^2', '430 ft^2', '434 ft^2',
       '545 ft^2', '303 ft^2', '419 ft^2', '310 ft^2', '345 ft^2',
       '290 ft^2', '1000 ft^2', '800 ft^2', '693 ft^2', '150 ft^2',
```

Untuk mengatasi masalah ini, satuan "ft^2" dihapus dari setiap entri menggunakan `*str.replace*`, dan frasa seperti "more than 1000" dipetakan ke nilai numerik 1001 menggunakan kamus `*dict_plinth_area*` dan fungsi `*apply*` dengan `l*ambda*`. Selanjutnya, tipe data variabel diubah menjadi float dan kemudian ke tipe data integer yang mendukung nilai NaN (`pd.Int64Dtype`).

```
train['plinth_area (ft^2)'] = train['plinth_area
(ft^2)'].str.replace(' ft^2', '')
test['plinth_area (ft^2)'] = test['plinth_area
(ft^2)'].str.replace(' ft^2', '')

dict_plinth_area = {
    'more than 1000': 1001
}
train['plinth_area (ft^2)'] = train['plinth_area
(ft^2)'].apply(lambda x: dict_plinth_area.get(x, x))
test['plinth_area (ft^2)'] = test['plinth_area
(ft^2)'].apply(lambda x: dict_plinth_area.get(x, x))
display(train['plinth_area (ft^2)'].value_counts(),
test['plinth_area (ft^2)'].value_counts())

# Mengubah ke tipe data integer (NaN tetap ada)
train['plinth_area (ft^2)'] = train['plinth_area
(ft^2)'].astype(float).astype(pd.Int64Dtype())
test['plinth_area (ft^2)'] = test['plinth_area
(ft^2)'].astype(float).astype(pd.Int64Dtype())

display(train['plinth_area (ft^2)'].describe(),
test['plinth_area (ft^2)'].describe())
```

```
plinth_area (ft^2)
300     8756
450     6724
400     6352
350     6071
1001    4922
        ...
971        1
994        1
797        1
849        1
93         1
Name: count, Length: 930, dtype: int64
plinth_area (ft^2)
300     2171
450     1730
350     1560
400     1555
1001    1320
        ...
766        1
767        1
781        1
881        1
905        1
Name: count, Length: 880, dtype: int64
```

Variabel '*type_of_foundation*'

Variabel '*type_of_foundation*' pada dataset train dan test memiliki variasi dalam penamaan jenis fondasi.

```
display(train['type_of_foundation'].unique(),
test['type_of_foundation'].unique())
```

```
array(['clay mortar-stone/brick', 'mud mortar-stone/brick',
       'clay sand mixed mortar-stone/brick', 'rc',
       'cement-stone or cement-brick', 'cement-stone/brick',
       'bamboo or timber', 'reinforced concrete', 'bamboo/timber', nan,
       'other', 'others'], dtype=object)
array(['clay mortar-stone/brick', 'clay sand mixed mortar-stone/brick',
       'rc', 'cement-stone/brick', nan, 'bamboo or timber',
       'bamboo/timber', 'reinforced concrete',
       'cement-stone or cement-brick', 'mud mortar-stone/brick', 'others',
       'other'], dtype=object)
```

Untuk mengatasi ini, dilakukan pemetaan penamaan yang seragam menggunakan kamus `*dict_type_foundation*`. Kamus ini mengganti *'cement-stone or cement-brick'* menjadi '*cement-stone/brick*', *'bamboo or timber'* menjadi *'bamboo/timber'*, dan '*other*' menjadi '*others*'. Pemetaan ini diterapkan menggunakan fungsi `*apply*` dengan `*lambda*`. Setelah proses ini, jumlah masing-masing nilai pada variabel ini ditampilkan untuk memastikan pemetaan yang benar.

```
dict_type_foundation = {
    'cement-stone or cement-brick': 'cement-stone/brick',
    'bamboo or timber': 'bamboo/timber',
    'other': 'others'
}
train['type_of_foundation'] =
train['type_of_foundation'].apply(lambda x:
dict_type_foundation.get(x, x))
test['type_of_foundation'] =
test['type_of_foundation'].apply(lambda x:
dict_type_foundation.get(x, x))
display(train['type_of_foundation'].value_counts(),
test['type_of_foundation'].value_counts())
```

```
type_of_foundation
clay sand mixed mortar-stone/brick    125447
clay mortar-stone/brick               100128
mud mortar-stone/brick                 25922
bamboo/timber                          24087
cement-stone/brick                     16333
rc                                      9063
reinforced concrete                     4218
others                                  1952
Name: count, dtype: int64
type_of_foundation
clay sand mixed mortar-stone/brick    31372
clay mortar-stone/brick               25045
mud mortar-stone/brick                 6565
bamboo/timber                          5981
cement-stone/brick                     4007
rc                                     2330
reinforced concrete                    1101
others                                  446
Name: count, dtype: int64
```

Variabel '*type_of_roof*'

Variabel *'type_of_roof'* pada dataset train dan test memiliki variasi dalam penamaan jenis atap.

```
display(train['type_of_roof'].unique(),
test['type_of_roof'].unique())
```

```
array(['wood light roof or bamboo heavy roof', 'bamboo/timber-light roof',
       nan, 'bamboo or timber light roof',
       'reinforced brick slab/rcc/rbc',
       'wood light roof or bamboo light roof', 'bamboo/timber light roof',
       'reinforced cement concrete/rb/rbc', 'bamboo or timber heavy roof',
       'bamboo/timber heavy roof', 'bamboo/timber-heavy roof',
       'rcc/rb/rbc', 'reinforced brick concrete/rcc/rbc'], dtype=object)
array(['wood light roof or bamboo light roof', nan,
       'wood light roof or bamboo heavy roof', 'bamboo/timber light roof',
       'bamboo or timber light roof', 'reinforced cement concrete/rb/rbc',
       'bamboo/timber-light roof', 'rcc/rb/rbc',
       'bamboo/timber heavy roof', 'bamboo or timber heavy roof',
       'bamboo/timber-heavy roof', 'reinforced brick concrete/rcc/rbc',
       'reinforced brick slab/rcc/rbc'], dtype=object)
```

Untuk mengatasi ini, dilakukan pemetaan penamaan yang seragam menggunakan kamus '*dict_type_roof*'. Kamus ini mengganti *'bamboo/timber-light roof'* dan *'bamboo or timber light roof'* menjadi *'bamboo/timber light roof'*, serta *'bamboo or timber heavy roof'* dan *'bamboo/timber-heavy roof'* menjadi *'bamboo/timber heavy roof'*. Pemetaan ini diterapkan menggunakan fungsi *apply* dengan *lambda*. Setelah proses ini, jumlah masing-masing nilai pada variabel ini ditampilkan untuk memastikan pemetaan yang benar.

```
dict_type_roof = {
    'bamboo/timber-light roof': 'bamboo/timber light roof',
    'bamboo or timber light roof': 'bamboo/timber light roof',
    'bamboo or timber heavy roof': 'bamboo/timber heavy roof',
    'bamboo/timber-heavy roof': 'bamboo/timber heavy roof'
}
train['type_of_roof'] = train['type_of_roof'].apply(lambda x:
dict_type_roof.get(x, x))
test['type_of_roof'] = test['type_of_roof'].apply(lambda x:
dict_type_roof.get(x, x))
display(train['type_of_roof'].value_counts(),
test['type_of_roof'].value_counts())
```

```
type_of_roof
bamboo/timber light roof                111797
wood light roof or bamboo heavy roof     55963
wood light roof or bamboo light roof     47789
bamboo/timber heavy roof                 11410
reinforced cement concrete/rb/rbc         8213
rcc/rb/rbc                                2842
reinforced brick slab/rcc/rbc             1966
reinforced brick concrete/rcc/rbc          997
Name: count, dtype: int64
type_of_roof
bamboo/timber light roof                27870
wood light roof or bamboo heavy roof    14100
wood light roof or bamboo light roof    12079
bamboo/timber heavy roof                 2943
reinforced cement concrete/rb/rbc        2102
rcc/rb/rbc                                711
reinforced brick slab/rcc/rbc             502
reinforced brick concrete/rcc/rbc         251
Name: count, dtype: int64
```

Variabel '*type_of_ground_floor*'

Variabel *'type_of_ground_floor'* pada dataset train dan test memiliki variasi dalam penamaan jenis lantai dasar.

```
display(train['type_of_ground_floor'].unique(),
test['type_of_ground_floor'].unique())
```

```
array(['clay', 'mud', nan, 'soil, water, loam mixed',
       'reinforced concrete', 'brick or stone', 'rc', 'brick/stone',
       'other', 'lumber', 'timber', 'wood'], dtype=object)
array(['clay', nan, 'mud', 'reinforced concrete', 'rc', 'brick or stone',
       'soil, water, loam mixed', 'lumber', 'brick/stone', 'other',
       'timber', 'wood'], dtype=object)
```

Untuk mengatasi ini, dilakukan pemetaan penamaan yang seragam menggunakan kamus '*dict_type_ground_floor*'. Kamus ini mengganti *'brick or stone'* menjadi *'brick/stone'*. Pemetaan ini diterapkan menggunakan fungsi *apply* dengan *lambda*. Setelah proses ini, jumlah masing-masing nilai pada variabel ini ditampilkan untuk memastikan pemetaan yang benar.

```
dict_type_ground_floor = {
    'brick or stone': 'brick/stone'
}
train['type_of_ground_floor'] =
train['type_of_ground_floor'].apply(lambda x:
dict_type_ground_floor.get(x, x))
test['type_of_ground_floor'] =
test['type_of_ground_floor'].apply(lambda x:
dict_type_ground_floor.get(x, x))
display(train['type_of_ground_floor'].value_counts(),test['type
_of_ground_floor'].value_counts())
```

```
type_of_ground_floor
clay                        189692
mud                          34611
brick/stone                  24530
reinforced concrete          15039
rc                           12128
soil, water, loam mixed       4524
lumber                        1079
other                          420
timber                         132
wood                            68
Name: count, dtype: int64
type_of_ground_floor
clay                        47544
mud                          8666
brick/stone                  6143
reinforced concrete          3769
rc                           3055
soil, water, loam mixed      1142
lumber                        278
other                          99
timber                         35
wood                           14
Name: count, dtype: int64
```

Variabel '*type_of_other_floor*'

Variabel *'type_of_other_floor'* pada dataset train dan test memiliki variasi dalam penamaan jenis lantai lainnya.

```
display(train['type_of_other_floor'].value_counts(),
test['type_of_other_floor'].value_counts())
```

```
type_of_other_floor
timber/bamboo-mud                    153919
not applicable                        46242
timber-planck                         21538
lumber-plank                          18971
wood-mud or bamboo mud                18915
rcc/rb/rbc                            11321
timber mud or bamboo-mud               9470
wood-plank                             7307
wood or bamboo mud                     3747
reinforced cement concrete/rb/rbc       635
reinforced brick concrete/rcc/rbc       513
Name: count, dtype: int64
type_of_other_floor
timber/bamboo-mud                    38631
not applicable                       11342
timber-planck                         5374
lumber-plank                          4832
wood-mud or bamboo mud                4704
rcc/rb/rbc                            2968
timber mud or bamboo-mud              2309
wood-plank                            1867
wood or bamboo mud                     957
reinforced cement concrete/rb/rbc      147
reinforced brick concrete/rcc/rbc      131
Name: count, dtype: int64
```

Untuk mengatasi ini, dilakukan pemetaan penamaan yang seragam menggunakan kamus *dict_type_of_other_floor*. Kamus ini mengganti *'timber mud or bamboo-mud'* menjadi *'timber/bamboo-mud'*, *'wood or bamboo mud'* dan *'wood-mud or bamboo mud'* menjadi *'wood/bamboo mud'*, *'reinforced cement concrete/rb/rbc'* dan *'reinforced brick concrete/rcc/rbc'* menjadi *'rcc/rb/rbc'*, serta *'timber-planck'* menjadi *'timber-plank'*. Pemetaan ini diterapkan menggunakan fungsi *apply* dengan *lambda*. Setelah proses ini, jumlah masing-masing nilai pada variabel ini ditampilkan untuk memastikan pemetaan yang benar.

```
# Menggabungkan kategori yang mirip
dict_type_of_other_floor = {
    'timber mud or bamboo-mud': 'timber/bamboo-mud',
    'wood or bamboo mud': 'wood/bamboo mud',
    'wood-mud or bamboo mud': 'wood/bamboo mud',
```

```
    'reinforced cement concrete/rb/rbc': 'rcc/rb/rbc',
    'reinforced brick concrete/rcc/rbc': 'rcc/rb/rbc',
    'timber-planck': 'timber-plank'
}

train['type_of_other_floor'] =
train['type_of_other_floor'].apply(lambda x:
dict_type_of_other_floor.get(x, x))
test['type_of_other_floor'] =
test['type_of_other_floor'].apply(lambda x:
dict_type_of_other_floor.get(x, x))
display(train['type_of_other_floor'].value_counts(),
test['type_of_other_floor'].value_counts())
```

```
type_of_other_floor
timber/bamboo-mud    163389
not applicable        46242
wood/bamboo mud       22662
timber-plank          21538
lumber-plank          18971
rcc/rb/rbc            12469
wood-plank             7307
Name: count, dtype: int64
type_of_other_floor
timber/bamboo-mud    40940
not applicable       11342
wood/bamboo mud       5661
timber-plank          5374
lumber-plank          4832
rcc/rb/rbc            3246
wood-plank            1867
Name: count, dtype: int64
```

Variabel '*legal_ownership_status*'

```
display(train['legal_ownership_status'].value_counts(),
test['legal_ownership_status'].value_counts())
```

```
legal_ownership_status
private              161397
private use          103958
privste               15418
prvt                  15399
public                 3363
public use             2949
public space           2152
institutional          1433
institutional use      1197
institutionals          839
other                   669
unknown                 610
unspecified             422
Name: count, dtype: int64
legal_ownership_status
private              40244
private use          25971
prvt                  4024
privste               3834
public                 819
public use             745
public space           546
institutional          340
institutional use      295
institutionals         218
other                  171
unknown                147
unspecified            110
Name: count, dtype: int64
```

Variabel *'legal_ownership_status'* pada dataset train dan test memiliki variasi dalam penamaan status kepemilikan yang berbeda-beda seperti *'private use'*, *'privste'*, dan *'prvt'* yang semuanya disesuaikan menjadi *'private'*. Sementara itu, istilah *'public use'* dan *'public space'* digabung menjadi *'public'*. Selain itu,

istilah *'institutional use'* dan *'institutionals'* diganti menjadi *'institutional'*. Istilah *'unknown'* dan *'unspecified'* dikelompokkan menjadi *'other'* untuk memperjelas status yang tidak dapat diidentifikasi secara pasti. Pemetaan ini dilakukan dengan menggunakan kamus *dict_legal_ownership_status* yang diterapkan melalui fungsi *apply* dan *lambda* pada kedua dataset. Setelah transformasi ini, jumlah masing-masing nilai pada variabel ini ditampilkan untuk memastikan pemetaan yang konsisten dan akurat.

```
dict_legal_ownership_status = {
    'private use': 'private',
    'privste': 'private',
    'prvt': 'private',
    'public use': 'public',
    'public space': 'public',
    'institutional use': 'institutional',
    'institutionals': 'institutional',
    'unknown': 'other',
    'unspecified': 'other'
}

train['legal_ownership_status'] =
train['legal_ownership_status'].apply(lambda x:
dict_legal_ownership_status.get(x, x))
test['legal_ownership_status'] =
test['legal_ownership_status'].apply(lambda x:
dict_legal_ownership_status.get(x, x))
display(train['legal_ownership_status'].value_counts(),
test['legal_ownership_status'].value_counts())
```

```
legal_ownership_status
private          296172
public             8464
institutional      3469
other              1701
Name: count, dtype: int64
legal_ownership_status
private          74073
public            2110
institutional      853
other              428
Name: count, dtype: int64
```

Variabel '*residential_type*'

```
display(train['residential_type'].value_counts(),
test['residential_type'].value_counts())
```

```
residential_type
non-residential          285275
hotel/motel               10794
rental residential         2448
housing                    1261
other                        80
other residential type       24
Name: count, dtype: int64
residential_type
non-residential          71261
hotel/motel               2724
rental residential         663
housing                    317
other                       36
other residential type       3
Name: count, dtype: int64
```

Variabel *'residential_type'* pada dataset train dan test menunjukkan variasi dalam penamaan tipe tempat tinggal, seperti *'other residential type'*, yang diganti menjadi *'other'* untuk konsistensi. Transformasi ini diterapkan menggunakan kamus *dict_residential_type* dengan fungsi *apply* dan *lambda* pada kedua dataset. Setelah proses ini, jumlah masing-masing nilai pada variabel ini ditampilkan untuk memastikan penamaan yang seragam dan kohesif.

```
dict_residential_type = {
    'other residential type': 'other'
}

train['residential_type'] =
train['residential_type'].apply(lambda x:
dict_residential_type.get(x, x))
test['residential_type'] =
test['residential_type'].apply(lambda x:
dict_residential_type.get(x, x))
display(train['residential_type'].value_counts(),
test['residential_type'].value_counts())
```

```
residential_type
non-residential       285275
hotel/motel            10794
rental residential      2448
housing                 1261
other                    104
Name: count, dtype: int64
residential_type
non-residential       71261
hotel/motel            2724
rental residential      663
housing                 317
other                    39
Name: count, dtype: int64
```

2.2.3.2 Feature Engineering

Langkah selanjutnya adalah melakukan *feature engineering*. Pada langkah ini dilakukan *feature encoding* dengan menggunakan LabelEncoder pada variabel technical_solution_proposed. Tujuannya adalah untuk mengubah variabel tersebut dari data kategorikal (string) menjadi data nominal (bilangan bulat). Proses *Feature engineering* dilakukan dengan *syntax* sebagai berikut:

```
# membuat objek label encoder
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()

# Encode
train['technical_solution_proposed'] =
le.fit_transform(train['technical_solution_proposed'])
test['technical_solution_proposed'] =
le.fit_transform(test['technical_solution_proposed'])
```

2.2.3.3 Missing Value

Langkah selanjutnya adalah melakukan pengisian *missing value*. Pada langkah ini, tahap pertama yang dilakukan adalah membagi data menjadi data numerik dan data kategorik sebagai berikut:

```
numerical_columns = ['floors_before_eq (total)',
'old_building', 'plinth_area (ft^2)', 'height_before_eq
(ft)','no_family_residing']

categorical_columns =
['has_secondary_use','wall_material','type_of_reinforcement_con
crete','land_surface_condition', 'type_of_foundation',
'type_of_roof', 'type_of_ground_floor','type_of_other_floor',
'position', 'building_plan_configuration',
'legal_ownership_status','residential_type',
'public_place_type','industrial_use_type',
'govermental_use_type','flexible_superstructure',
'wall_binding']
```

Selanjutnya dilakukan pengisian *missing value* dengan menggunakan SimpleImputer, yaitu imputasi median untuk data numerik dan imputasi modus untuk data kategorik. Pengisian *missing value* dengan imputasi median untuk variabel numerik dilakukan dengan *syntax* sebagai berikut:

```
#Penanganan data numerik
from sklearn.impute import SimpleImputer
for col in numerical_columns:
    imputer = SimpleImputer(strategy='median')
    train[[col]] = imputer.fit_transform(train[[col]])
    test[[col]] = imputer.fit_transform(test[[col]])
```

Pengisian *missing value* dengan imputasi modus untuk variabel kategorik dilakukan dengan *syntax* sebagai berikut:

```
#Penanganan data kategorik
for col in categorical_columns:
    imputer = SimpleImputer(strategy='most_frequent')
    train[[col]] = imputer.fit_transform(train[[col]])
    test[[col]] = imputer.fit_transform(test[[col]])
```

Selain itu, pengisian *missing value* juga dilakukan pada variabel technical_solution_proposed dengan menggunakan KNNImputer. KNNImputer merupakan metode pengisian *missing value* berdasarkan nilai-nilai terdekat dari data lainnya yang tidak memiliki *missing value*. Imputasi KNNImputer dilakukan dengan *syntax* sebagai berikut:

```
from sklearn.impute import KNNImputer
imputer = KNNImputer()
train[['technical_solution_proposed']] =
imputer.fit_transform(train[['technical_solution_proposed']])
test[['technical_solution_proposed']] =
imputer.fit_transform(test[['technical_solution_proposed']])
```

2.2.3.4 Encoding

Langkah selanjutnya adalah melakukan *feature encoding* dengan menggunakan OneHotEncoder pada beberapa variabel kategorik sebagai berikut:

```
onehot_cols = ['land_surface_condition', 'type_of_foundation',
'type_of_roof', 'type_of_ground_floor', 'type_of_other_floor',
'position', 'building_plan_configuration',
'technical_solution_proposed', 'legal_ownership_status',
'has_secondary_use', 'type_of_reinforcement_concrete',
'residential_type', 'public_place_type',
'industrial_use_type', 'govermental_use_type',
'flexible_superstructure', 'wall_material', 'wall_binding']
```

Feature encoding dilakukan untuk mengubah variabel kategorik menjadi variabel dummy menggunakan *syntax* sebagai berikut:

Data train:

```
# Membuat instance dari OneHotEncoder tanpa menyebutkan
'sparse'
from sklearn.preprocessing import OneHotEncoder
onehot_encoder = OneHotEncoder()

# Melakukan fit dan transformasi pada kolom 'embarked'
train_onehot = onehot_encoder.fit_transform(train[onehot_cols])

# Mengonversi hasilnya menjadi DataFrame
train_onehot_df = pd.DataFrame(train_onehot.toarray(),
columns=onehot_encoder.get_feature_names_out(onehot_cols))

# Joining the numerical and categorical dfframes
train_encoded = pd.concat([train['ID'],
train[numerical_columns],
train_onehot_df,  train['damage_grade']], axis=1)

train_encoded
```

Data test:

```
# Membuat instance dari OneHotEncoder tanpa menyebutkan
'sparse'
from sklearn.preprocessing import OneHotEncoder
onehot_encoder = OneHotEncoder()

# Melakukan fit dan transformasi pada kolom 'embarked'
test_onehot = onehot_encoder.fit_transform(test[onehot_cols])

# Mengonversi hasilnya menjadi DataFrame
test_onehot_df = pd.DataFrame(test_onehot.toarray(),
columns=onehot_encoder.get_feature_names_out(onehot_cols))

# Joining the numerical and categorical dfframes
test_encoded = pd.concat([test['ID'], test[numerical_columns],
test_onehot_df], axis=1)


test_encoded
```

Berikut merupakan contoh hasil dari OneHotEncoder pada variabel land_surface_conditional yang memiliki sebanyak 3 kategori:

land_surface_condition_flat	land_surface_condition_moderate slope	land_surface_condition_steep slope
1.0	0.0	0.0
0.0	1.0	0.0
1.0	0.0	0.0
1.0	0.0	0.0
0.0	1.0	0.0
...	...	...
1.0	0.0	0.0
1.0	0.0	0.0
1.0	0.0	0.0
1.0	0.0	0.0
1.0	0.0	0.0

Karena variabel land_surface_condition memiliki 3 kategori (*flat*, *moderate slope*, dan *steep slope*), maka penerapan OneHotEncoder pada variabel ini akan menghasilkan tiga variabel baru yang menggantikan variabel asli land_surface_condition. Setiap variabel baru ini akan bernilai 1 jika kategori pada baris tersebut sesuai dan akan bernilai 0 jika tidak sesuai.

2.2.3.5 Standarisasi

Langkah selanjutnya adalah melakukan standarisasi dengan menggunakan MinMaxScaler. Standarisasi ini bertujuan untuk mengubah skala fitur-fitur numerik sehingga berada dalam rentang yang sama, yaitu pada rentang antara 0 dan 1. Standarisasi dilakukan dengan menggunakan *syntax* sebagai berikut:

```
from sklearn.preprocessing import MinMaxScaler
# Inisialisasi MinMaxScaler
scaler = MinMaxScaler()
```

```
# Fit dan transform data train
train_encoded[numerical_columns] =
scaler.fit_transform(train_encoded[numerical_columns])

# Transform data test dengan parameter yang sama dari data
train
test_encoded[numerical_columns] =
scaler.transform(test[numerical_columns])
```

2.2.4 Splitting Data

Setelah melakukan beberapa tahapan *preprocessing data*, maka langkah berikutnya adalah membagi data train menjadi dua bagian, yaitu X (variabel prediktor) dan y (variabel respon damage_grade).

```
X = train_encoded.drop(['damage_grade', 'ID'],axis = 1)
y = train_encoded['damage_grade']
```

Langkah berikutnya adalah membagi data train yang telah dipisahkan menjadi X dan y ke dalam dua set, yaitu data train dan data validasi. Pembagian ini dilakukan dengan menggunakan rasio 8:2. *Splitting data* dilakukan dengan menggunakan *syntax* sebagai berikut:

```
# Split data menjadi train dan val
from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size =
0.2, random_state = 26)
```

2.2.5 SMOTE

Setelah melakukan *splitting data*, maka langkah berikutnya adalah menangani ketidakseimbangan kelas dalam dataset. Salah satu pendekatan yang dapat digunakan untuk menangani ketidakseimbangan kelas adalah dengan menggunakan SMOTE (*Synthetic Minority Over-sampling Technique*). SMOTE dilakukan dengan menggunakan *syntax* sebagai berikut.

```
from imblearn.over_sampling import SMOTE
# Inisialisasi objek SMOTE
smote = SMOTE(random_state=21725)

# Melakukan oversampling pada data
X_resampled, y_resampled = smote.fit_resample(X_train,y_train)
```

Data hasil SMOTE dapat digunakan untuk tahap pemodelan data.

2.2.6 Pemodelan

Setelah dilakukan serangkaian *pre processing* data, akan dilakukan pemodelan prediktif dengan beberapa algoritma klasifikasi sebagai berikut:

2.2.6.1 Cat Boost

Pemodelan algoritma CatBoost dilakukan dengan syntax sebagai berikut:

```
cat_clf = CatBoostClassifier()
cat_clf.fit(X_resampled, y_resampled)
cat_pred = cat_clf.predict(X_val)
metrics = classification_report(y_val, cat_pred, digits=4)
print("\n",metrics)
```

Output:

```
    accuracy                         0.4671     62002
   macro avg     0.4380    0.4164    0.4040     62002
weighted avg     0.4400    0.4671    0.4273     62002
```

Berdasarkan hasil di atas, diperoleh nila F1 score macro avg sebesar 0,4040 untuk algoritma Cat Boost. Ini menunjukkan bahwa, secara umum model belum memiliki kinerja yang baik sehingga perlu evaluasi lebih lanjut untuk mencapai hasil yang lebih baik. Selain itu, diperoleh juga nilai akurasi sebesar 0,4671 yang menunjukkan bahwa model Cat Boost mampu memprediksi dengan benar sebanyak 46,71% data. Nilai akurasi ini tergolong masih belum cukup baik.

2.2.6.2 Hist Gradient Boosting

Pemodelan algoritma Hist Gradient Boosting dilakukan dengan syntax sebagai berikut:

```
hgbc_sm_clf = HistGradientBoostingClassifier()
hgbc_sm_clf.fit(X_resampled, y_resampled)
hgbc_sm_pred = hgbc_sm_clf.predict(X_val)
metrics = classification_report(y_val, hgbc_sm_pred, digits=4)
print("\n",metrics)
```

Output:

```
              precision    recall  f1-score   support

           1     0.5194    0.6608    0.5816      6754
           2     0.3625    0.2056    0.2624      7227
           3     0.4001    0.2121    0.2773     11314
           4     0.3665    0.1824    0.2436     14837
           5     0.4896    0.8042    0.6086     21870

    accuracy                         0.4620     62002
   macro avg     0.4276    0.4130    0.3947     62002
weighted avg     0.4322    0.4620    0.4175     62002
```

Berdasarkan hasil di atas, diperoleh nila F1 score macro avg sebesar 0,3947 untuk algoritma Hist Gradient Boosting. Ini menunjukkan bahwa, secara umum model belum memiliki kinerja yang baik sehingga perlu evaluasi lebih lanjut untuk mencapai hasil yang lebih baik. Selain itu, diperoleh juga nilai akurasi sebesar 0,4620 yang menunjukkan bahwa model Hist Gradient Boosting mampu memprediksi dengan benar sebanyak 46,20% data. Nilai akurasi ini tergolong masih belum cukup baik.

2.2.6.3 Random Forest

Pemodelan algoritma Random Forest dilakukan dengan syntax sebagai berikut:

```
rf_sm_clf = RandomForestClassifier()
rf_sm_clf.fit(X_resampled, y_resampled)
rf_sm_pred = rf_sm_clf.predict(X_val)
metrics = classification_report(y_val, rf_sm_pred, digits=4)
print("\n",metrics)
```

Output:

```
              precision    recall  f1-score   support

           1     0.5008    0.5802    0.5376      6754
           2     0.2775    0.2657    0.2715      7227
           3     0.3045    0.2733    0.2880     11314
           4     0.3229    0.2926    0.3070     14837
           5     0.5109    0.5526    0.5309     21870

    accuracy                         0.4090     62002
   macro avg     0.3833    0.3929    0.3870     62002
weighted avg     0.3999    0.4090    0.4035     62002
```

Berdasarkan hasil di atas, diperoleh nila F1 score macro avg sebesar 0,3870 untuk algoritma Random Forest. Ini menunjukkan bahwa, secara umum model belum memiliki kinerja yang baik sehingga perlu evaluasi lebih lanjut untuk mencapai hasil yang lebih baik. Selain itu, diperoleh juga nilai akurasi sebesar 0,4090 yang menunjukkan bahwa model Random Forest mampu memprediksi dengan benar sebanyak 40,90% data. Nilai akurasi ini tergolong masih belum cukup baik.

2.2.6.4 Gradient Boosting

Pemodelan algoritma Gradient Boosting dilakukan dengan syntax sebagai berikut:

```
gb_sm_clf = GradientBoostingClassifier()
gb_sm_clf.fit(X_resampled, y_resampled)
gb_sm_pred = gb_sm_clf.predict(X_val)
metrics = classification_report(y_val, gb_sm_pred, digits=4)
print("\n",metrics)
```

Output:

```
              precision    recall  f1-score   support

           1     0.4909    0.6757    0.5687      6754
           2     0.3260    0.2017    0.2492      7227
           3     0.3429    0.2566    0.2935     11314
           4     0.3406    0.2645    0.2977     14837
           5     0.5096    0.6582    0.5745     21870

    accuracy                         0.4394     62002
   macro avg     0.4020    0.4113    0.3967     62002
weighted avg     0.4153    0.4394    0.4184     62002
```

Berdasarkan hasil di atas, diperoleh nila F1 score macro avg sebesar 0,3967 untuk algoritma Gradient Boosting. Ini menunjukkan bahwa, secara umum model belum memiliki kinerja yang baik sehingga perlu evaluasi lebih lanjut untuk mencapai hasil yang lebih baik. Selain itu, diperoleh juga nilai akurasi sebesar 0,4394 yang menunjukkan bahwa model Gradient Boosting mampu memprediksi dengan benar sebanyak 43,94% data. Nilai akurasi ini tergolong masih belum cukup baik.

2.2.6.5 Extra Trees Classifier

Pemodelan algoritma Extra Trees Classifier dilakukan dengan syntax sebagai berikut:

```
etc_sm_clf = ExtraTreesClassifier()
etc_sm_clf.fit(X_resampled, y_resampled)
etc_sm_pred = etc_sm_clf.predict(X_val)
metrics = classification_report(y_val, etc_sm_pred, digits=4)
print("\n",metrics)
```

Output:

```
              precision    recall  f1-score   support

           1     0.4814    0.5345    0.5066      6754
           2     0.2585    0.2680    0.2632      7227
           3     0.2911    0.2808    0.2859     11314
           4     0.3111    0.2927    0.3016     14837
           5     0.5017    0.5078    0.5047     21870

    accuracy                         0.3899     62002
   macro avg     0.3688    0.3768    0.3724     62002
weighted avg     0.3871    0.3899    0.3882     62002
```

Berdasarkan hasil di atas, diperoleh nila F1 score macro avg sebesar 0,3724 untuk algoritma Extra Trees Classifier. Ini menunjukkan bahwa, secara umum model belum memiliki kinerja yang baik sehingga perlu evaluasi lebih lanjut untuk mencapai hasil yang lebih baik. Selain itu, diperoleh juga nilai akurasi

sebesar 0,3899 yang menunjukkan bahwa model Extra Trees Classifier mampu memprediksi dengan benar sebanyak 38,99% data. Nilai akurasi ini tergolong masih belum cukup baik.

2.2.6.6 MLP Classifier

Pemodelan algoritma MLP Classifier dilakukan dengan syntax sebagai berikut:

```
mlp_clf = MLPClassifier()
mlp_clf.fit(X_resampled, y_resampled)
mlp_pred = mlp_clf.predict(X_val)
metrics = classification_report(y_val,mlp_pred, digits=4)
print("\n",metrics)
```

Output:

```
              precision    recall  f1-score   support

           1     0.4910    0.5967    0.5387      6754
           2     0.2593    0.3185    0.2859      7227
           3     0.3267    0.2897    0.3071     11314
           4     0.3455    0.2772    0.3076     14837
           5     0.5302    0.5570    0.5433     21870

    accuracy                         0.4178     62002
   macro avg     0.3905    0.4078    0.3965     62002
weighted avg     0.4130    0.4178    0.4133     62002
```

Berdasarkan hasil di atas, diperoleh nila F1 score macro avg sebesar 0,3965 untuk algoritma MLP Classifier. Ini menunjukkan bahwa, secara umum model belum memiliki kinerja yang baik sehingga perlu evaluasi lebih lanjut untuk mencapai hasil yang lebih baik. Selain itu, diperoleh juga nilai akurasi sebesar 0,4178 yang menunjukkan bahwa model MLP Classifier mampu memprediksi dengan benar sebanyak 41,78% data. Nilai akurasi ini tergolong masih belum cukup baik.

2.2.6.7 AdaBoost Classifier

Pemodelan algoritma AdaBoost Classifier dilakukan dengan syntax sebagai berikut:

```
ada_sm_clf = AdaBoostClassifier()
ada_sm_clf.fit(X_resampled, y_resampled)
ada_sm_pred = ada_sm_clf.predict(X_val)
metrics = classification_report(y_val, ada_sm_pred, digits=4)
print("\n",metrics)
```

Output:

```
              precision    recall  f1-score   support

           1     0.4576    0.6593    0.5402      6754
           2     0.2463    0.2053    0.2240      7227
           3     0.3327    0.2534    0.2877     11314
           4     0.3167    0.3352    0.3257     14837
           5     0.5141    0.5155    0.5148     21870

    accuracy                         0.4040     62002
   macro avg     0.3735    0.3938    0.3785     62002
weighted avg     0.3964    0.4040    0.3970     62002
```

Berdasarkan hasil di atas, diperoleh nila F1 score macro avg sebesar 0,3785 untuk algoritma AdaBoost Classifier. Ini menunjukkan bahwa, secara umum model belum memiliki kinerja yang baik sehingga perlu evaluasi lebih lanjut untuk mencapai hasil yang lebih baik. Selain itu, diperoleh juga nilai akurasi sebesar 0,4040 yang menunjukkan bahwa model AdaBoost Classifier mampu memprediksi dengan benar sebanyak 40,40% data. Nilai akurasi ini tergolong masih belum cukup baik.

2.2.6.8 Model Terbaik

Dari beberapa pemodelan yang telah dilakukan, diperoleh bahwa model Cat Boost memiliki nilai F1 Score macro avg yang paling tinggi dibandingkan model yang lain. Oleh karena itu dilakukan prediksi data test menggunakan model Cat Boost dengan syntax sebagai berikut:

```
cek = test_encoded.copy()
cek.set_index(['ID'], inplace = True)
cek['damage_grade'] = cat_clf.predict(cek)
cek['ID'] = cek.index
cat = cek[['ID', 'damage_grade']]
cat
```

Diperoleh hasil prediksi data test sebagai berikut:

	ID	damage_grade
ID		
71840	71840	5
85155	85155	4
287066	287066	4
124951	124951	5
73581	73581	5
...	...	...
238185	238185	3
290899	290899	5
184490	184490	5
342028	342028	5
386710	386710	3

77503 rows × 2 columns

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan permasalahan yang terdapat dalam laporan ini, dilakukan analisis prediktif dampak yang diterima oleh suatu bangunan akibat gempa bumi berdasarkan spesifikasinya menggunakan algoritma klasifikasi. Beberapa algoritma yang digunakan dalam laporan ini, yaitu Cat Boost, Hist Gradient Boosting, Random Forest, Gradient Boosting, Extra Trees Classifier, MLP Classifier, dan AdaBoost Classifier. Sebelum dilakukan pemodelan, dilakukan serangkaian *pre processing* data, yaitu penanganan inkonsisten data, *feature engineering* (mengubah data technical_solution_proposed ke dalam data nominal), penanganan *missing value*, *feature encoding* (mengubah data kategorik menjadi variabel dummy), standarisasi data numerik, dan SMOTE. Pemodelan dilakukan dengan membagi data train menjadi 80% train dan 20% val, diperoleh bahwa algoritma Cat Boost memiliki nilai F1 Score macro avg tertinggi dibanding model lain yang dianalisis pada laporan ini. Pada model Cat Boost, diperoleh nilai akurasi sebesar 0,4671, F1 Score macro avg sebesar 0,4040, dan F1 score weighted avg sebesar 0,4273.

3.2 Saran

Sebelumnya, kami ingin mengucapkan terima kasih kepada asisten praktikum yang telah membantu kami dalam pembelajaran praktikum Komputasi Statistika II. Terima kasih atas ilmu yang telah disampaikan selama setengah semester ini. Asisten praktikum telah melaksanakan tugasnya dengan baik. Penyampaian materinya juga sudah jelas. Untuk saran sejauh ini belum ada, karena sudah cukup baik.